

LAPORAN PRAKTIKUM

Message Passing Interface

Laporan ini untuk memenuhi salah satu tugas Mata Kuliah

Komputasi Paralel dan Terdistribusi

Dosen Pengampu: Ir. Randi Rizal, Ph.D.



Disusun Oleh:

Mochammad Fasha Ajvikri (247006111178)

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS SILIWANGI

2026

1. Tugas Praktikum 1 (SCATTER SEDERHANA)

1.1 Tujuan

- Memahami distribusi data dari rank 0 ke seluruh proses menggunakan Scatter.
- Mengamati perilaku urutan output pada eksekusi paralel

1.2 Spesifikasi

- Proses 0 memiliki list [10, 20, 30, 40].
- Data dibagi rata ke 4 proses dengan scatter.
- Setiap proses mencatat data yang diterimanya.

1.3 Kode Program

```
# Nama: Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Pastikan jumlah proses = 4
if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

data = [10, 20, 30, 40] if rank == 0 else None
recv = comm.scatter(data, root=0)

print(f"Rank {rank} menerima {recv}")
```

1.4 Output

```
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 4 python 1.py
Rank 2 menerima 30
Rank 0 menerima 10
Rank 1 menerima 20
Rank 3 menerima 40
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> █
```

1.5 Tabel Hasil

Rank	Data Diterima
0	10
1	20
2	30
3	40

Pertanyaan Diskusi

- Apa output yang dihasilkan tiap proses?

Setiap proses dengan rank 0, 1, 2, dan 3 masing-masing menerima tepat satu elemen data secara terurut dari list asal pada root (rank 0), yaitu rank 0 menerima 10, rank 1 menerima 20, rank 2 menerima 30, dan rank 3 menerima 40.

- Mengapa urutannya bisa berbeda saat dicetak?

Hal ini disebabkan oleh sifat *non-deterministic* dari eksekusi proses paralel dan adanya mekanisme *buffering* pada *standard output* (stdout). Karena setiap proses MPI berjalan secara independen dan bersamaan, waktu akses pencetakan ke layar tergantung pada penjadwalan CPU oleh sistem operasi.

2. Tugas Praktikum 2 (Gather Sederhana)

2.1 Tujuan

- Memahami pengumpulan data dari semua proses ke root menggunakan fungsi Gather.
- Mengamati perubahan hasil saat jumlah proses dikembangkan.

2.2 Spesifikasi

- Tiap proses menyiapkan data lokal berupa kalkulasi rank * 5.
- Data dikumpulkan kembali di proses 0 dan dicetak dalam bentuk list terurut.

2.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Setiap proses menghitung data lokalnya sendiri
# berdasarkan ID rank
send_val = rank * 5

# Mengumpulkan (gather) nilai dari SELURUH proses
# kembali ke rank 0.
# Hasil dikembalikan dalam bentuk list Python di rank
# 0, sedangkan di rank lain bernilai None
gathered = comm.gather(send_val, root=0)
# Cetak hasil pengumpulan hanya di proses utama (rank
# 0)
if rank == 0:
    print(f"Hasil gather di rank 0 (n={size}):
    {gathered}")
```

2.4 Output

```
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 4 python 2.py
Hasil gather di rank 0 (n=4): [0, 5, 10, 15]
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 6 python 2.py
Hasil gather di rank 0 (n=6): [0, 5, 10, 15, 20, 25]
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> █
```

2.5 Tabel Hasil

Jumlah Proses	Hasil list di rank
4	[0, 5, 10, 15]
6	[0, 5, 10, 15, 20, 25]

Pertanyaan Diskusi

- **Apa isi list hasil di proses 0 jika ada 4 proses?**
Isi list-nya adalah [0, 5, 10, 15], yang merupakan kumpulan nilai rank * 5 dari rank 0 hingga rank 3.
- **Bagaimana hasilnya jika jumlah proses = 6?**
List hasil berkembang menyesuaikan jumlah proses menjadi [0, 5, 10, 15, 20, 25], memuat nilai dari rank 0 hingga rank 5.
- **Jelaskan mengapa urutan elemen sesuai rank.**
Fungsi comm.gather secara internal menyusun data penerimaan berdasarkan nomor ID rank pengirim secara terurut (dari rank 0 sampai \$n-1\$), bukan berdasarkan siapa yang paling cepat mengirimkan datanya ke jaringan.

3. Tugas Praktikum 3 (Scatter + Gather Kombinasi)

3.1 Tujuan

- Menggabungkan distribusi data (scatter), komputasi lokal, dan pengumpulan hasil (gather).
- Memvalidasi transformasi elemen (kuadrat) per-proses.

3.2 Spesifikasi

- Proses 0 memiliki list [1, 2, 3, 4].
- Data di-scatter ke 4 proses.
- Setiap proses menghitung nilai kuadrat dari datanya.
- Hasil di-gather kembali ke proses 0 (uji juga untuk list [2, 4, 6, 8]).

3.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

#
-----

# Uji Coba 1: Input List [1, 2, 3, 4]
#
-----

data1 = [1, 2, 3, 4] if rank == 0 else None
```

```

# Step 1: Distribusi data awal dari rank 0 ke seluruh
proses
x1 = comm.scatter(data1, root=0)

# Step 2: Komputasi lokal independen di masing-masing
core/proses
y1 = x1 * x1

# Step 3: Pengumpulan kembali hasil komputasi lokal
ke rank 0
hasil1 = comm.gather(y1, root=0)

if rank == 0:
    print(f"Input : {data1}")
    print(f"Output: {hasil1}\n")

#
-----
-----

# Uji Coba 2: Input List [2, 4, 6, 8]
#
-----
-----

data2 = [2, 4, 6, 8] if rank == 0 else None
x2 = comm.scatter(data2, root=0)
y2 = x2 * x2
hasil2 = comm.gather(y2, root=0)

if rank == 0:
    print(f"Input : {data2}")
    print(f"Output: {hasil2}")

```

3.4 Output

```
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 4 python 3.py
Input : [1, 2, 3, 4]
Output: [1, 4, 9, 16]

Input : [2, 4, 6, 8]
Output: [4, 16, 36, 64]
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> █
```

3.5 Tabel Hasil

Rank	Input [1, 2, 3, 4]	Output (Kuadrat)	Input [2, 4, 6, 8]	Output (Kuadrat)
0	1	1	2	4
1	2	4	4	16
2	3	9	6	32
3	4	16	8	64

Pertanyaan Diskusi

- **Apa hasil akhir yang dikumpulkan di proses 0 untuk [1, 2, 3, 4]?**

Hasil akhirnya adalah list [1, 4, 9, 16].

- **Bagaimana jika list awal diganti [2, 4, 6, 8]?**

Hasil akhirnya berubah menjadi list [4, 16, 36, 64].

- **Jelaskan korespondensi indeks (rank → posisi data).**

Proses scatter memetakan elemen array indeks ke-i tepat ke proses dengan rank == i. Setelah transformasi kuadrat selesai, proses gather menyusun kembali elemen dari rank == i ke posisi indeks ke-i pada array output akhir di rank 0.

4. Tugas Praktikum 4 (Hitung Jumlah dengan Scatter + Gather)

4.1 Tujuan

- Menerapkan transformasi lokal dan agregasi total nilai di rank 0.
- Melatih validasi komputasi terdistribusi.

4.2 Spesifikasi

- Proses 0 memiliki list [5, 10, 15, 20].
- Data di-scatter ke 4 proses.
- Tiap proses mengalikan datanya dengan 2 ($2 * x$).
- Hasil di-gather lalu dijumlahkan di proses 0.

4.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

# Data asal hanya ada di rank 0
data = [5, 10, 15, 20] if rank == 0 else None

# Distribusi elemen list ke tiap proses
x = comm.scatter(data, root=0)

# Transformasi lokal: kalikan data yang diterima
dengan 2
local = 2 * x

# Kumpulkan seluruh hasil perkalian ke rank 0
kumpul = comm.gather(local, root=0)
```

```
# Rank 0 melakukan agregasi sum() pada list hasil
gather
if rank == 0:
    total = sum(kumpul)
    print(f>Data awal      : {data}")
    print(f"x2 per-proses  : {kumpul}")
    print(f>Total akhir    : {total}")
```

4.4 Output

```
• (Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 4 python 4.py
Data awal      : [5, 10, 15, 20]
x2 per-proses  : [10, 20, 30, 40]
Total akhir    : 100
○ (Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> █
```

4.5 Tabel Hasil

Rank	Input	x2
0	5	10
1	10	20
2	15	30
3	20	40

Jawaban Singkat (Validasi)

Total = 100 karena $(2 \times 5) + (2 \times 10) + (2 \times 15) + (2 \times 20) = 10 + 20 + 30 + 40$.

5. Tugas Praktikum 5 (Reduce vs Allreduce)

5.1 Tujuan

- Membedakan Reduce (hasil agregasi hanya di root) vs Allreduce (hasil diketahui semua proses).
- Menggunakan MPI.MAX untuk pencarian nilai terbesar & MPI.SUM untuk kalkulasi total/rata-rata global.
- Mengagregasi data acak skala besar tanpa mengirim seluruh isi array.

5.2 Spesifikasi

- Bagian A (MAX): Cari nilai terbesar dari $\text{rank} * 7 + 3$ menggunakan Reduce.
- Bagian B (AVG): Hitung rata-rata global = (global sum) / size menggunakan Allreduce.
- Bagian C (Total 1000 acak/proses): Gunakan Allreduce untuk menghitung total global.

5.3 Kode Program

```
# Nama : Mochammad Fasha Ajvikri
# NPM : 247006111178
# Kelas: F

import numpy as np
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

#
-----
-----

# Bagian A: MAX dengan Reduce (Hasil hanya diterima
di rank 0)
#
-----
-----
```

```

my_val = rank * 7 + 3
# reduce() melakukan operasi pencarian nilai maksimum
(MPI.MAX)
max_val = comm.reduce(my_val, op=MPI.MAX, root=0)

if rank == 0:
    print(f"[REDUCE-MAX] Max value: {max_val}")

#
-----
-----
# Bagian B: Rata-rata Global via Allreduce (Hasil
diterima SEMUA rank)
#
-----
-----
val_for_avg = rank + 1
# allreduce() menghitung total penjumlahan seluruh
rank tanpa butuh root
global_sum = comm.allreduce(val_for_avg, op=MPI.SUM)
global_avg = global_sum / size

# Karena pakai Allreduce, setiap rank bisa mencetak
nilai global ini
print(
    f"[ALLREDUCE-AVG] Rank {rank}: sum={global_sum},
avg={global_avg}"
)

#
-----
-----
# Bagian C: Total 1000 Bilangan Acak / Proses via
Allreduce
#
-----
-----
# Buat generator angka acak unik per rank menggunakan
seed
rng = np.random.default_rng(seed=12345 + rank)

```

```

nums = rng.random(1000)

# Hitung jumlah lokal (1000 angka) di masing-masing
proses dulu
local_sum = float(np.sum(nums))

# Gabungkan total dari semua proses via Allreduce
total_sum = comm.allreduce(local_sum, op=MPI.SUM)

if rank == 0:
    print(
        f"[ALLREDUCE-SUM] Total 1000*{size} bilangan
acak: {total_sum:.6f}"
    )

```

5.4 Output

```

(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3> mpiexec -n 4 python 5.py
[ALLREDUCE-AVG] Rank 1: sum=10, avg=2.5
[ALLREDUCE-AVG] Rank 2: sum=10, avg=2.5
[REDUCE-MAX] Max value: 24
[ALLREDUCE-AVG] Rank 0: sum=10, avg=2.5
[ALLREDUCE-SUM] Total 1000*4 bilangan acak: 1993.195672
[ALLREDUCE-AVG] Rank 3: sum=10, avg=2.5
(Praktikum 3) PS D:\KULIAH\SMT 5\Komputasi Paralel dan Terdistribusi\Praktikum 3>

```

5.5 Tabel Hasil

Bagian	Metrik	Nilai
A	MAX (Reduce)	24
B	SUM (Allreduce)	10
B	AVG (SUM/Size)	2.5
C	TOTAL (Allreduce)	1993.195672

Pertanyaan Diskusi

- **Jelaskan perbedaan konseptual & praktis Reduce vs Allreduce**
Konseptual & Praktis: Reduce mengumpulkan data dari seluruh proses, melakukan operasi matematika agregat (SUM, MAX, MIN), dan hanya menyimpan hasil akhirnya di proses root (rank 0). Sementara Allreduce menjalankan operasi reduksi yang sama,

tetapi membagikan hasil akhirnya ke seluruh proses yang terlibat tanpa membutuhkan parameter root.

- **Mengapa MAX (fungsi naik terhadap rank) memunculkan nilai terbesar di rank terbesar?**

Fungsi perhitungan yang digunakan adalah $f(\text{rank}) = 7 \cdot \text{rank} + 3$. Karena koefisien variabel rank bernilai positif, fungsi ini bersifat *monotonically increasing* (nilainya terus bertambah seiring membesarnya ID rank). Sehingga nilai terbesar otomatis dihasilkan oleh rank dengan indeks paling tinggi (rank $n-1$).

- **Mengapa Allreduce efisien untuk agregasi besar (mis. 1000 bilangan acak/proses) dibanding mengirim array mentah?**

Allreduce memanfaatkan algoritma reduksi pohon (*tree reduction*) atau cincin (*ring reduction*). Setiap proses mengagregasi 1000 data lokalnya terlebih dahulu dan hanya mengirimkan satu angka skalar hasil penjumlahan melalui jaringan. Cara ini jauh lebih hemat lalu lintas data (*bandwidth*) daripada mengirim seluruh array $1000 \times n$ elemen mentah ke root.

Kesimpulan

- a. Fungsi komunikasi kolektif Scatter dan Gather memudahkan pola distribusi data *one-to-all* dan pengumpulan kembali *all-to-one* dalam eksekusi terdistribusi secara terstruktur.
- b. Penggunaan Reduce sangat efisien jika hasil agregasi hanya dibutuhkan oleh node utama (root), sedangkan Allreduce lebih praktis ketika seluruh node memerlukan angka agregat global tersebut untuk komputasi tahap selanjutnya.
- c. Mengolah komputasi di tingkat lokal sebelum melakukan agregasi kolektif terbukti memangkas *overhead* transmisi jaringan, sehingga efisiensi dan *throughput* sistem komputasi paralel meningkat pesat.